

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Technology Hits · [Follow publication](#)

Database queries in Python — SQLAlchemy-PostGRESQI

10 min read · Mar 26, 2023



Python Code Nemesis

Follow



Listen



Share



More

Introduction to PostgreSQL

PostgreSQL is a powerful open-source object-relational database management system (ORDBMS) that is widely used for managing large amounts of data. It is known for its reliability, stability, and robustness, making it a popular choice for organizations and applications that require high levels of data integrity and security.

PostgreSQL supports a variety of data types, including integers, floats, text, and dates, and provides many advanced features such as support for stored procedures, triggers, and views. It also has a strong emphasis on SQL compliance, providing support for complex queries and advanced SQL features such as window functions and common table expressions.

PostgreSQL is highly extensible, with a robust extension system that allows developers to add new functionality to the database easily. It is also cross-platform compatible, running on Windows, macOS, Linux, and many other operating systems.

SQLAlchemy

SQLAlchemy is an open-source Python library that provides a set of high-level tools and utilities for working with relational databases using Python. It allows developers to interact with a variety of SQL databases using a unified API, which abstracts away the differences between the various database engines.

SQLAlchemy provides an object-relational mapper (ORM) that allows developers to work with databases using Python objects rather than raw SQL queries. This makes it easier to manage the database schema and data, as well as providing a more Pythonic way of working with the database.

SQLAlchemy also provides a SQL expression language, which allows developers to write SQL queries in Python syntax. This makes it easier to write complex queries and manipulate database data using Python code.

In addition, SQLAlchemy provides support for connection pooling, transactions, and other common database management features. It also supports a variety of SQL databases, including PostgreSQL, MySQL, SQLite, and Oracle.

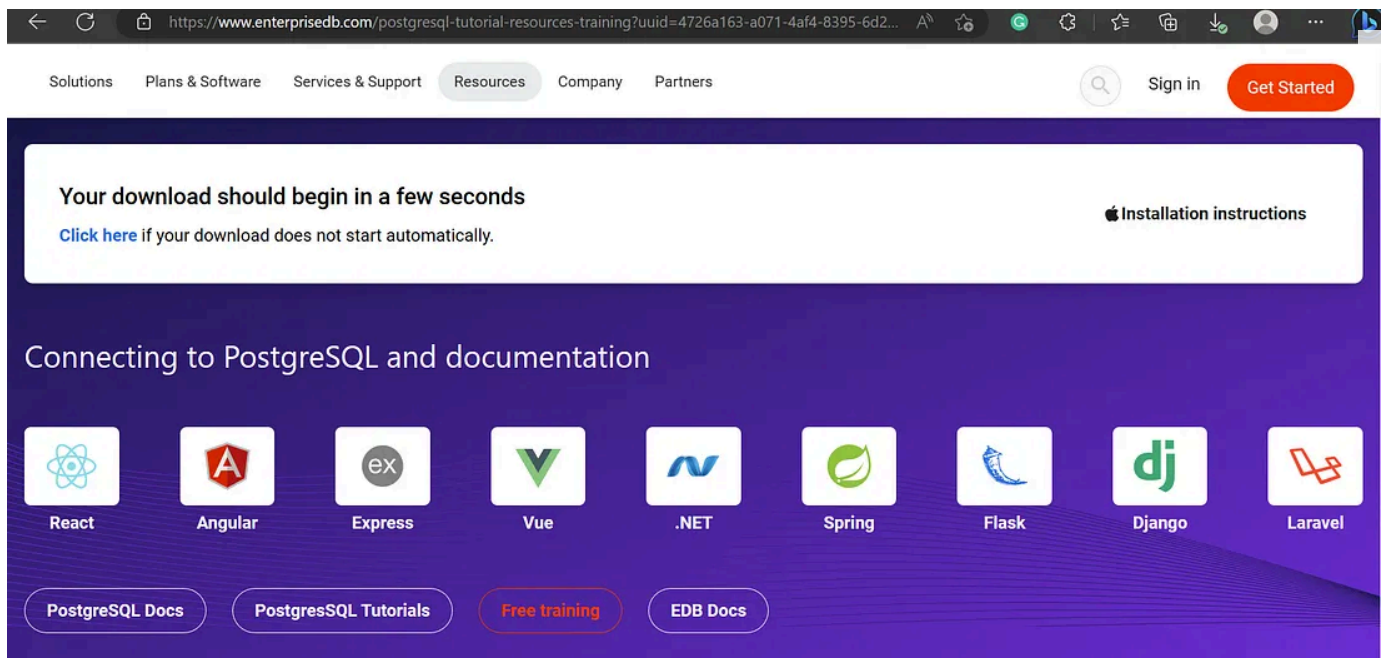
Database queries in Python — SQLAlchemy-PostGRESQI

Install PostgreSQL: You can download and install PostgreSQL from the official website for your operating system. Once installed, you should be able to start the PostgreSQL server.

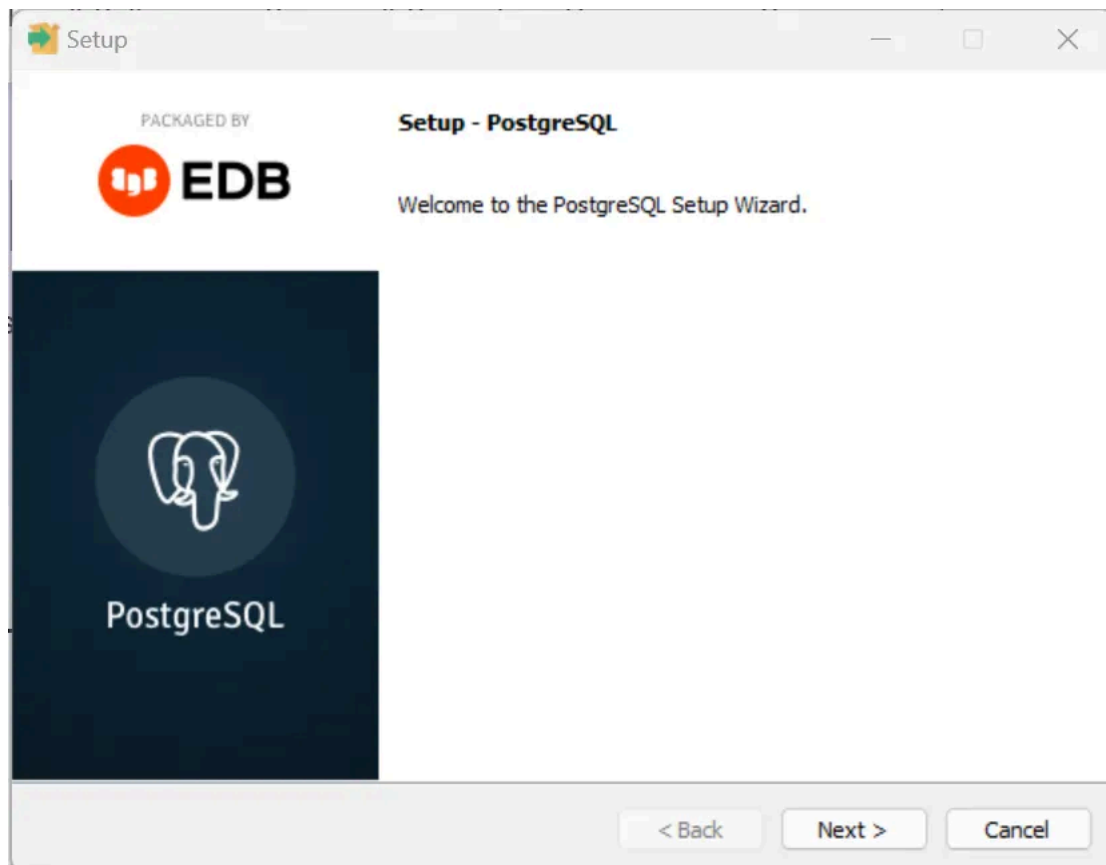
Community DL Page

www.enterprisedb.com

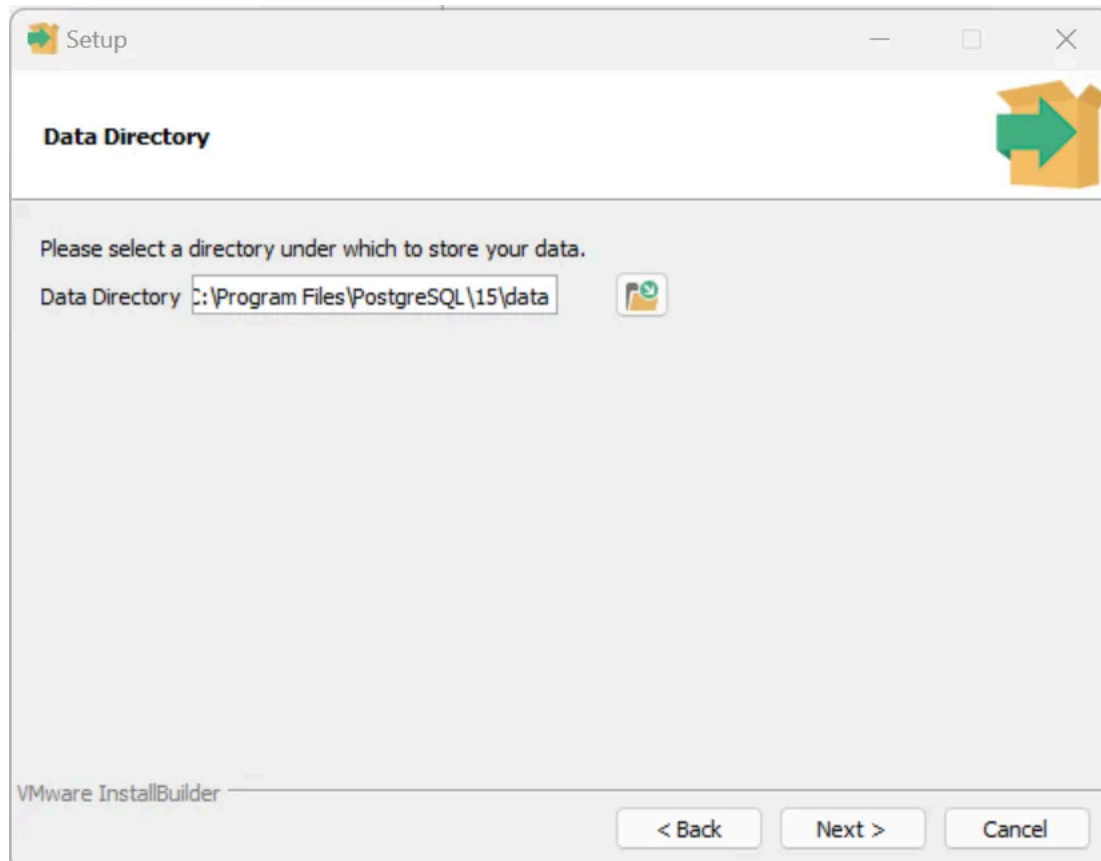
Download PostgreSQL 15:



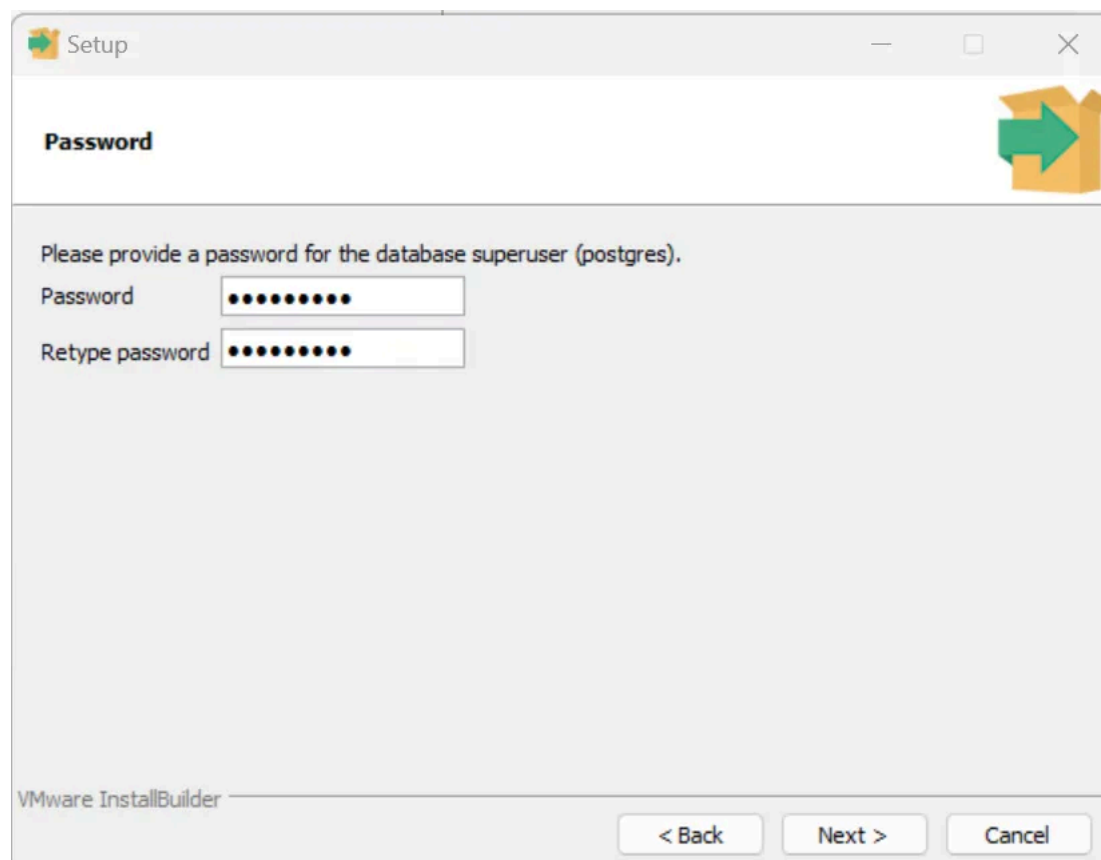
Start the setup:



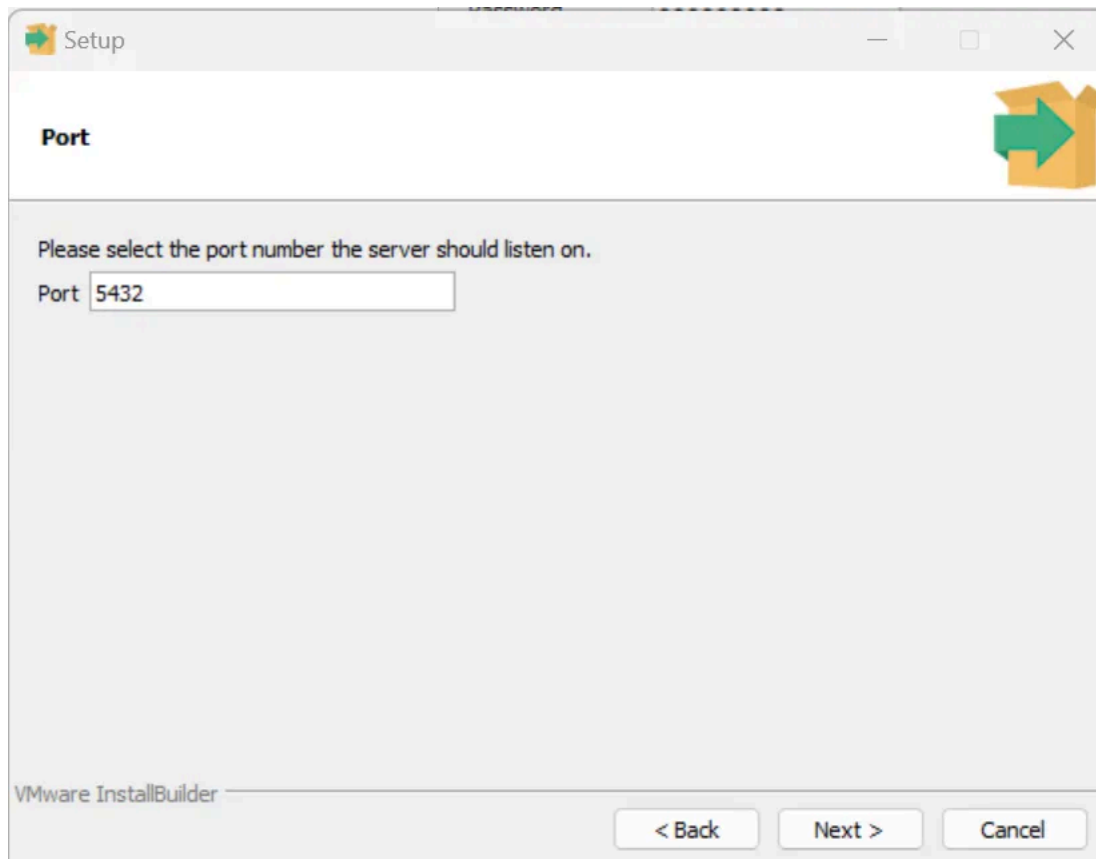
Choose the data directory:



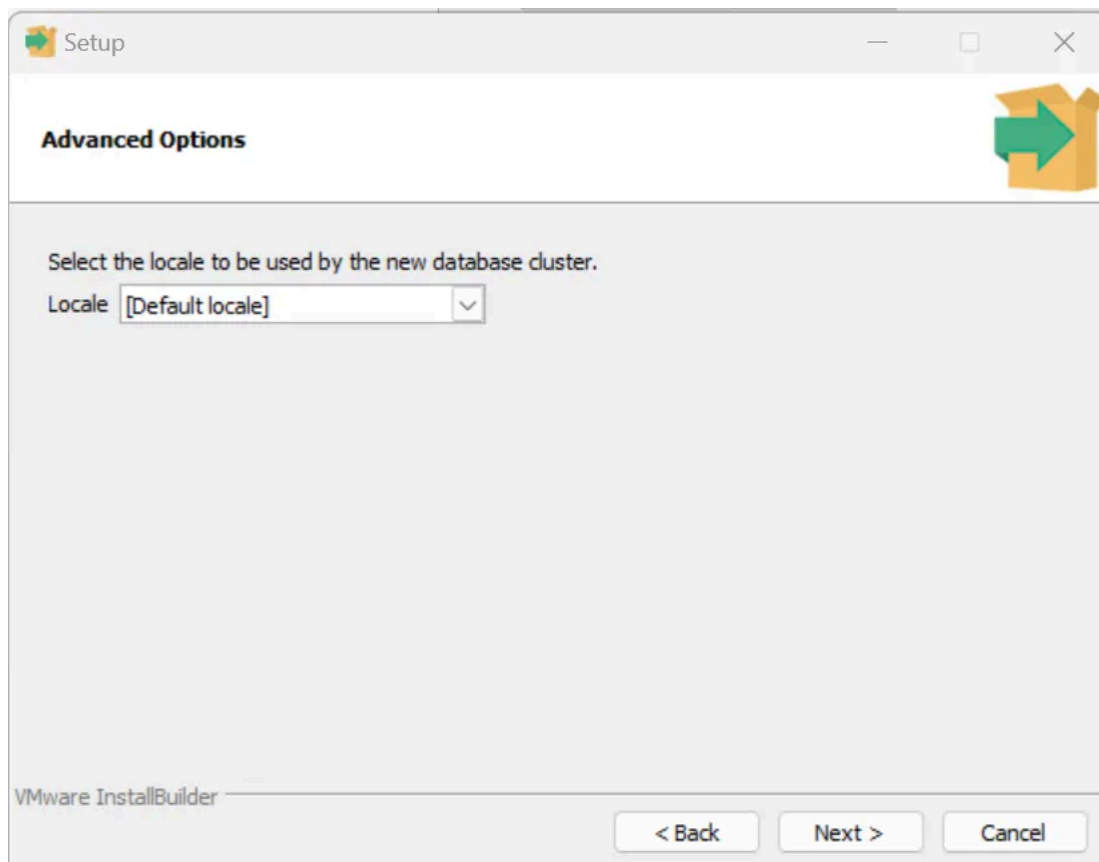
Provide a password and retype password:



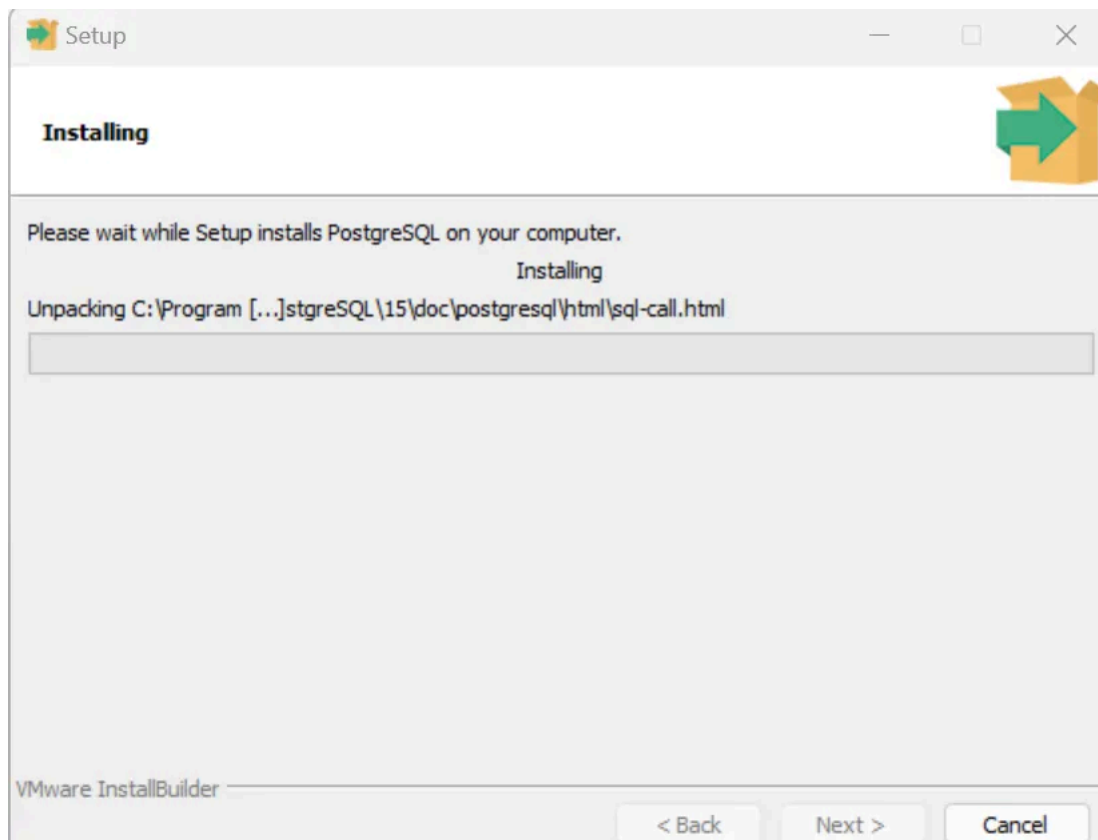
You can give any superuser password, eg Admin1234. Next select a port Number :



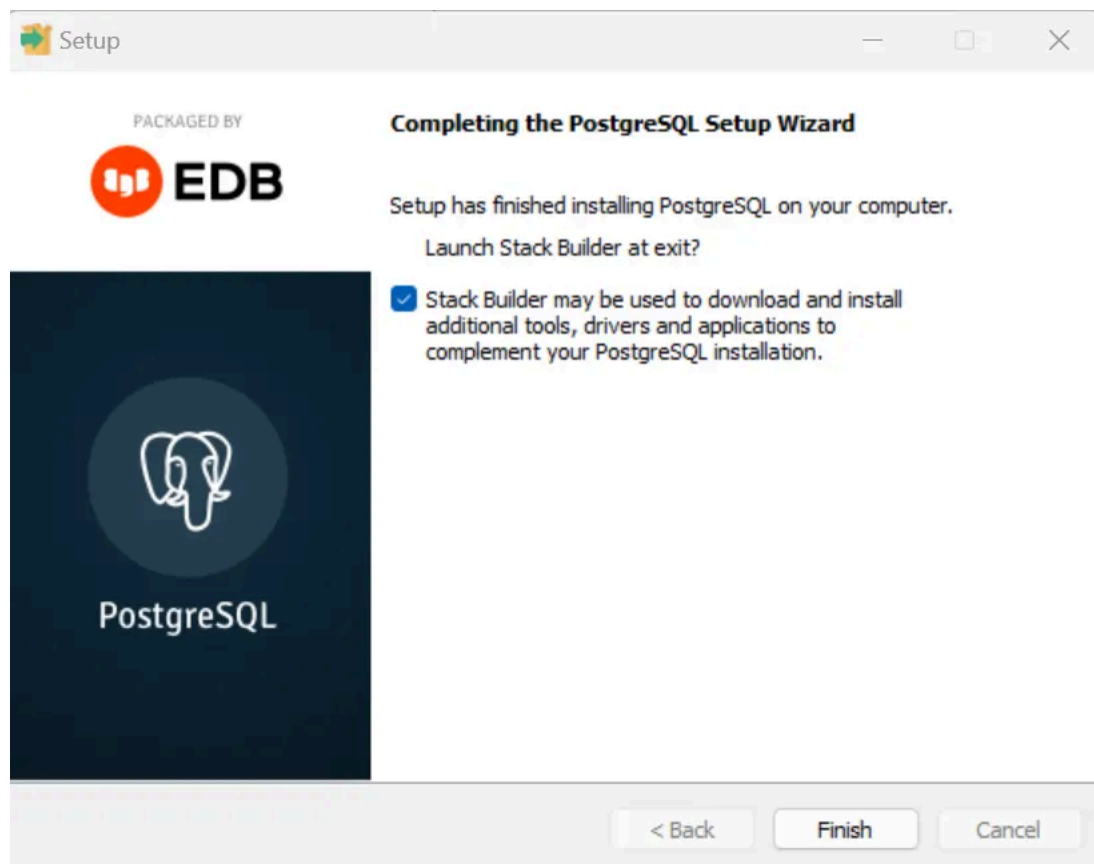
Next select locale. what is locale?



The install begins:



The Setup is done!



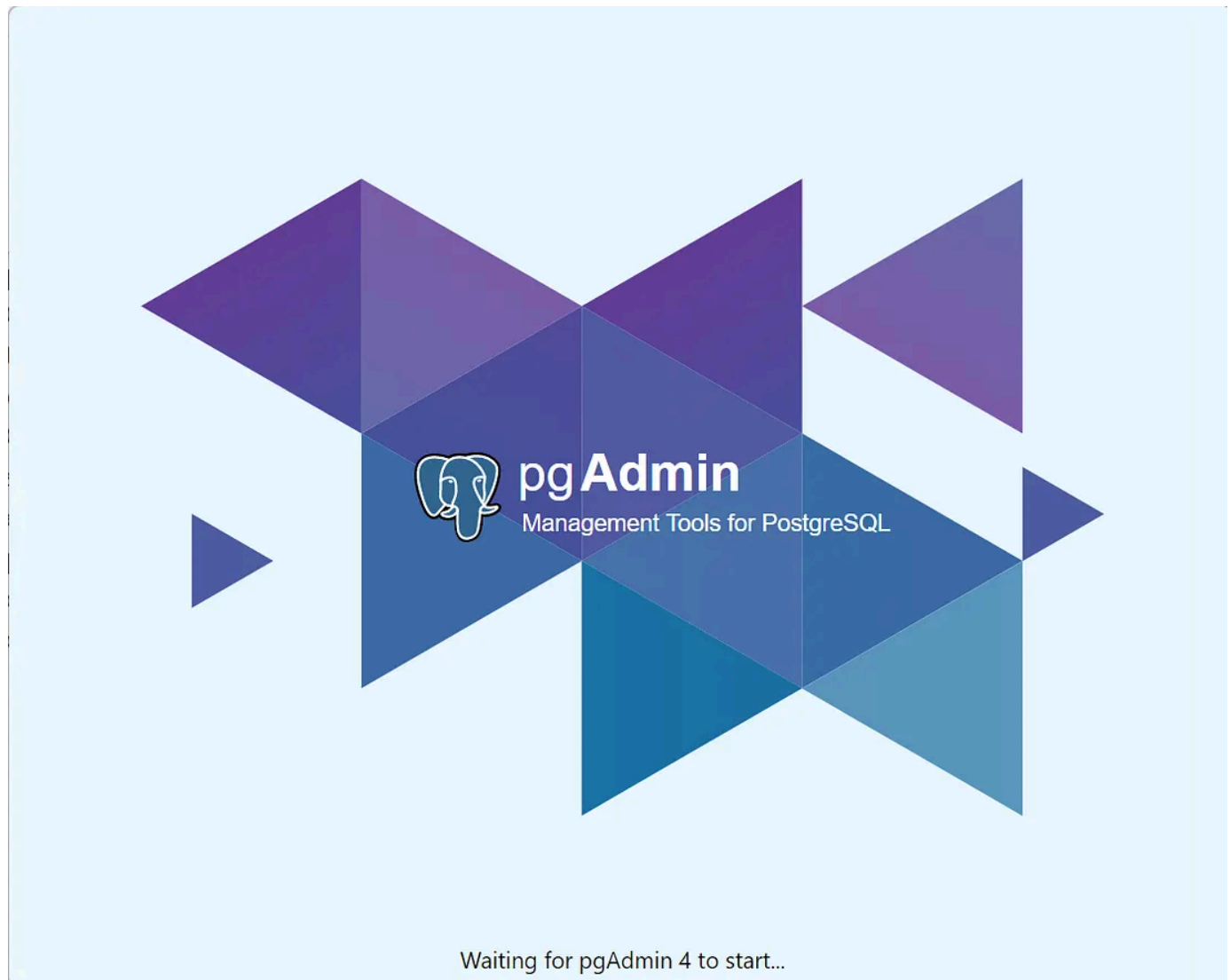
PGAdmin4

PgAdmin 4 is a free and open-source administration and management tool for PostgreSQL. It provides a graphical user interface (GUI) for managing and interacting with PostgreSQL databases, allowing users to create and manage databases, tables, views, users, and other database objects.

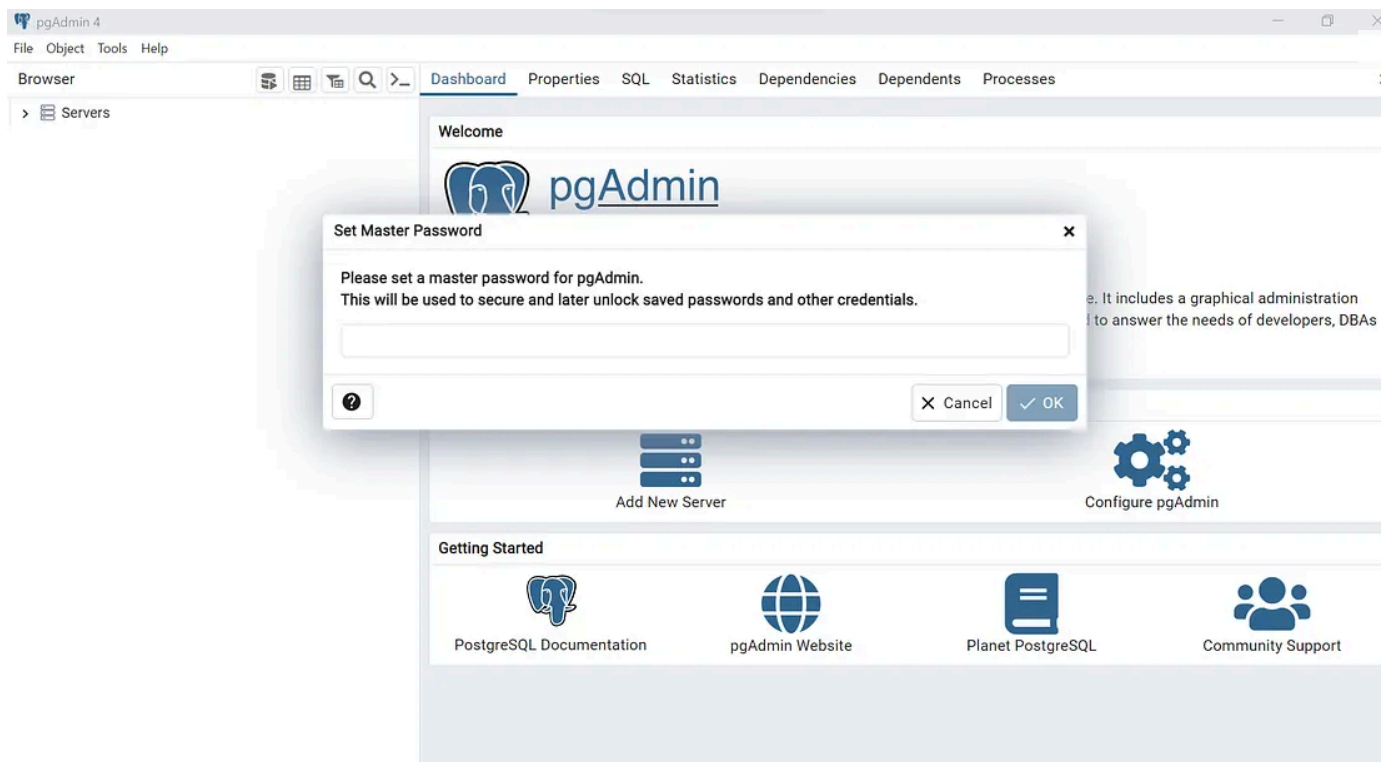
PgAdmin 4 offers a user-friendly interface that simplifies the process of managing PostgreSQL databases. It provides features such as syntax highlighting, code completion, and error highlighting for SQL queries, making it easier for users to write and debug SQL code. It also allows users to create and execute queries using a visual query builder tool.

In addition to database management, pgAdmin 4 provides tools for server administration, including server settings, server groups, and server logs. It also offers support for backup and restore operations, allowing users to easily backup and restore their databases.

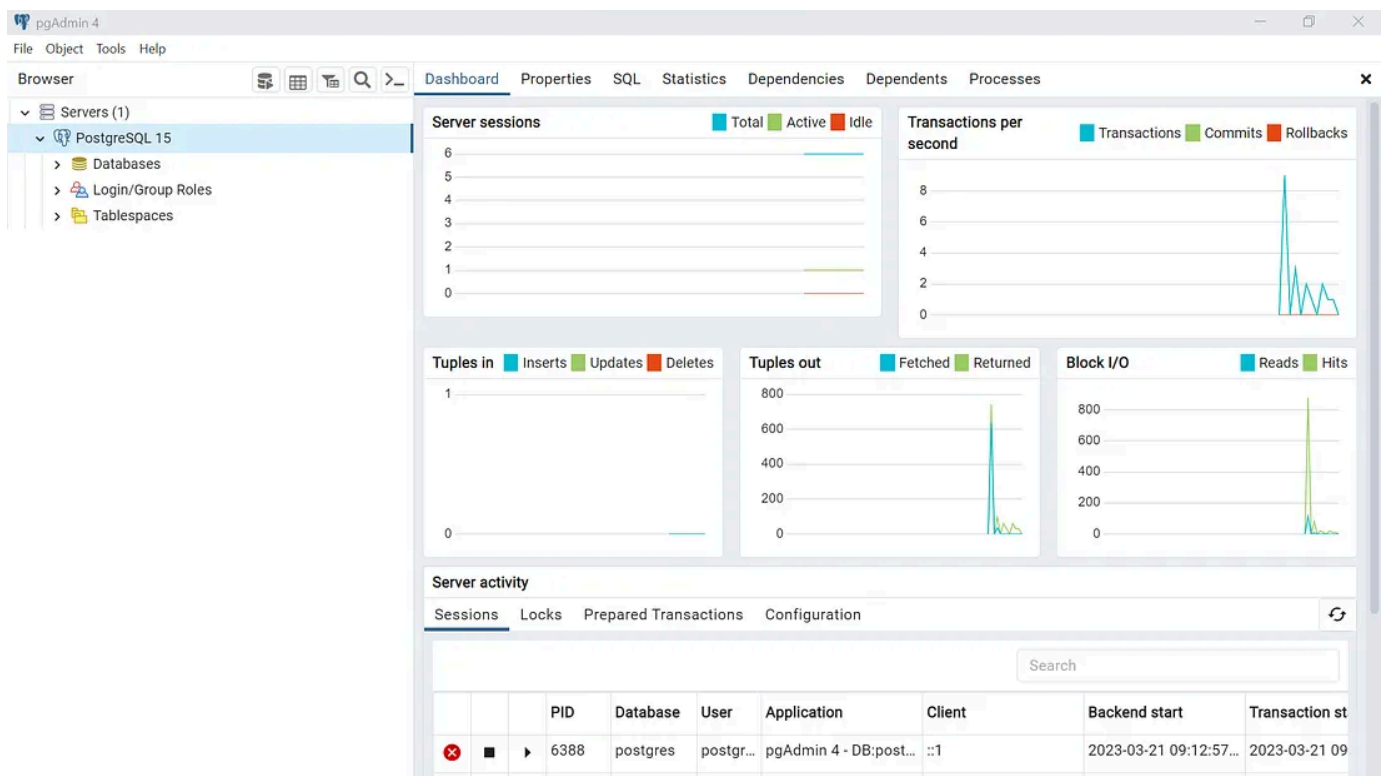
PgAdmin 4 is available for Windows, macOS, and Linux operating systems and can be downloaded for free from the official pgAdmin website. Open pgadmin4:



Set a Master Password the first time you open PGAdmin4, eg admin1234:



Enter postgres password :



Creating a new user

Create a new user: By default, PostgreSQL creates a user called `postgres` with full privileges. To create a new user, open a terminal and enter the following command:

```
createuser --interactive
```

Follow the prompts to create a new user with a name and password of your choice. Make sure to grant this user the `CREATEDB` privilege so that they can create new databases.

Create a new database: Once you have a user, you can create a new database for this tutorial. Open a terminal and enter the following command:

```
createdb mydatabase -O myuser
```

Replace `mydatabase` with the name of your new database, and `myuser` with the name of the user you created in step 2. This will create a new database with your user as the owner.

Connect to the database

To connect to your new database, open a terminal and enter the following command:

```
psql -d mydatabase -U myuser
```

Replace `mydatabase` with the name of your new database, and `myuser` with the name of the user you created in step 2. You will be prompted for the password for this user.

Get the database connection details: Once you're connected to the database, you can get the database connection details by entering the following SQL command:

```
SELECT current_user, current_database();
```

This will return the username and database name that you can use to connect to your PostgreSQL server.

That's it! You should now have a working PostgreSQL installation with a new database that you can use for this tutorial.

Connection details

To get your PostgreSQL database connection details, you will need to know the following information:

1. Host: This is the hostname or IP address of the machine where your PostgreSQL server is running. If your PostgreSQL server is running on the same machine as your application, you can use `localhost` or `127.0.0.1` as the host.
2. Port: This is the port number on which your PostgreSQL server is listening for connections. The default port number for PostgreSQL is `5432`.
3. Username: This is the username that you use to connect to your PostgreSQL server. By default, the `postgres` user is created during installation, but you can create additional users if needed.
4. Password: This is the password that you use to authenticate with your PostgreSQL server. If you're using a local installation of PostgreSQL, you may not have set a password for the `postgres` user.
5. Database name: This is the name of the database that you want to connect to.

You can usually find these details in your PostgreSQL configuration files or by asking your database administrator. If you're using a cloud-hosted PostgreSQL service, your provider should provide you with these details.

Once you have your database connection details, you can use them to connect to your PostgreSQL server using a PostgreSQL client or command-line interface like `psql`.

Create the users table in postgres database

To create the `users` table in a PostgreSQL database, you can use the following SQL query:

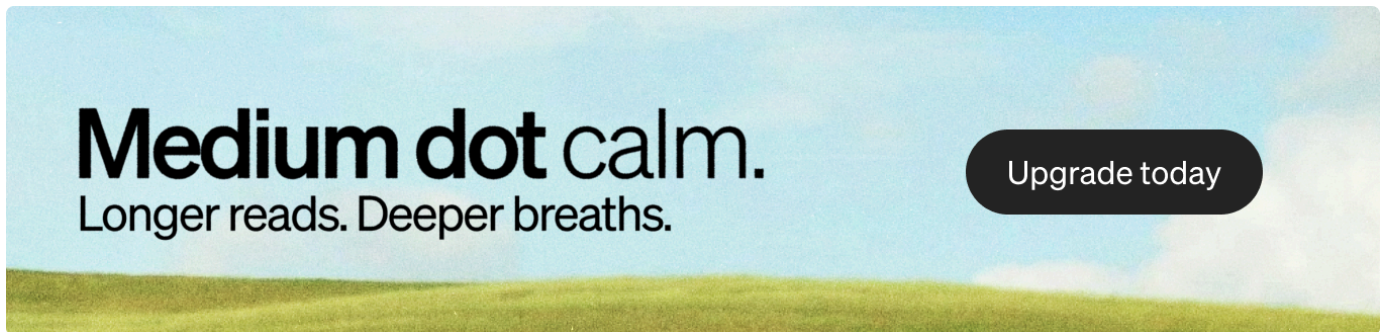
```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,
```



```
email VARCHAR(255) NOT NULL,  
age INT NOT NULL  
);
```

This will create a table called `users` with four columns: `id`, `name`, `email`, and `age`. The `id` column is a serial primary key, which means it will automatically increment with each new row added to the table. The `name`, `email`, and `age` columns are all required fields and cannot be null.

You can execute this SQL query using any PostgreSQL client or command line interface, such as `psql` or pgAdmin. Here's an example of how you can use the `psql` command line interface to execute the query:



Open a terminal window and enter the following command to connect to your PostgreSQL database:

```
psql -h <host> -p <port> -U <username> <database>
```

Replace `<host>`, `<port>`, `<username>`, and `<database>` with your actual database connection details.

Once you're connected to the database, enter the following SQL query to create the `users` table:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL,  
  age INT NOT NULL  
);
```

Press Enter to execute the query. If the query is successful, you should see a message similar to this:

```
CREATE TABLE
```

That's it! Your `users` table should now be created in your PostgreSQL database.

Writing SQL queries in SQL Alchemy

Next, we will write SQL queries in SQL Alchemy using a sample PostgreSQL table. We'll cover simple operations like `select`, `order by`, `distinct`, and `where`, and then move on to more complex queries like `joins` and `nested joins`.

Let's assume we have the following sample table called `users` in our PostgreSQL database:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(255) NOT NULL,  
  email VARCHAR(255) NOT NULL,
```

```
    age INT NOT NULL
);
```

This table has four columns: `id`, `name`, `email`, and `age`. `id` is a serial primary key, which means it automatically increments with each new row added to the table. `name`, `email`, and `age` are all required fields.

Now let's move on to writing SQL queries in SQL Alchemy.

First, we'll need to import the necessary modules:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker
from sqlalchemy.ext.declarative import declarative_base
```

Next, we'll create an engine object and connect to our PostgreSQL database:

```
engine = create_engine('postgresql://user:password@localhost:5432/database_name')
Session = sessionmaker(bind=engine)
session = Session()
```

Now that we're connected to the database, let's start with some simple SQL queries.

```
Base = declarative_base()

class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String)
    email = Column(String)
    age = Column(Integer)
```

SELECT

To select all columns from the `users` table, we can use the following code:

```
users = session.query(User).all()
for user in users:
    print(user.id, user.name, user.email, user.age)
```

This will output a list of all the users in the table, along with their ID, name, email, and age.

To select specific columns, we can use the `with_entities()` method:

```
users = session.query(User.id, User.name).all()
for user in users:
    print(user.id, user.name)
```

This will output only the ID and name columns for each user.

ORDER BY

To order the results by a specific column, we can use the `order_by()` method:

```
users = session.query(User).order_by(User.age).all()
for user in users:
    print(user.id, user.name, user.email, user.age)
```

This will output all the users in the table, ordered by age.

DISTINCT

To select only distinct values from a column, we can use the `distinct()` method:

```
users = session.query(User.age).distinct().all()
for user in users:
    print(user.age)
```

This will output all the distinct ages in the table.

WHERE

To select only certain rows based on a condition, we can use the `filter()` method:

```
users = session.query(User).filter(User.age > 30).all()
for user in users:
    print(user.id, user.name, user.email, user.age)
```

This will output all the users in the table who are older than 30. Now let's move on to more complex queries.

Here's the complete code for all the queries. Create a file called `queries.py` and insert the following:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import sessionmaker
from sqlalchemy.orm import declarative_base

engine = create_engine('postgresql://postgres:Admin1234@localhost:5432/postgres')
Session = sessionmaker(bind=engine)
session = Session()

Base = declarative_base()

class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String)
    email = Column(String)
    age = Column(Integer)

# SELECT QUERY 1
print("SELECT QUERY 1")
users = session.query(User).all()
for user in users:
    print(user.id, user.name, user.email, user.age)

# SELECT QUERY 2
print("SELECT QUERY 2")
users = session.query(User.id, User.name).all()
for user in users:
    print(user.id, user.name)

# ORDER BY
print("ORDER BY")
users = session.query(User).order_by(User.age).all()
for user in users:
    print(user.id, user.name, user.email, user.age)

# DISTINCT
```

```
print("DISTINCT")
users = session.query(User.age).distinct().all()
for user in users:
    print(user.age)

# WHERE
print("WHERE")
users = session.query(User).filter(User.age > 30).all()
for user in users:
    print(user.id, user.name, user.email, user.age)
```

The table before you execute this query will look like this:

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree is expanded to show 'PostgreSQL 15' and its 'Databases' (postgres). The 'Query' tab is active, showing the query 'SELECT * FROM users;'. Below the query, the 'Data Output' tab displays a single row of data:

id	name	email	age
1	Python Code Nemesis	pythoncodenemesis@gmail.com	23

The status bar at the bottom indicates 'Total rows: 1 of 1', 'Query complete 00:00:00.573', and 'Ln 1, Col 21'.

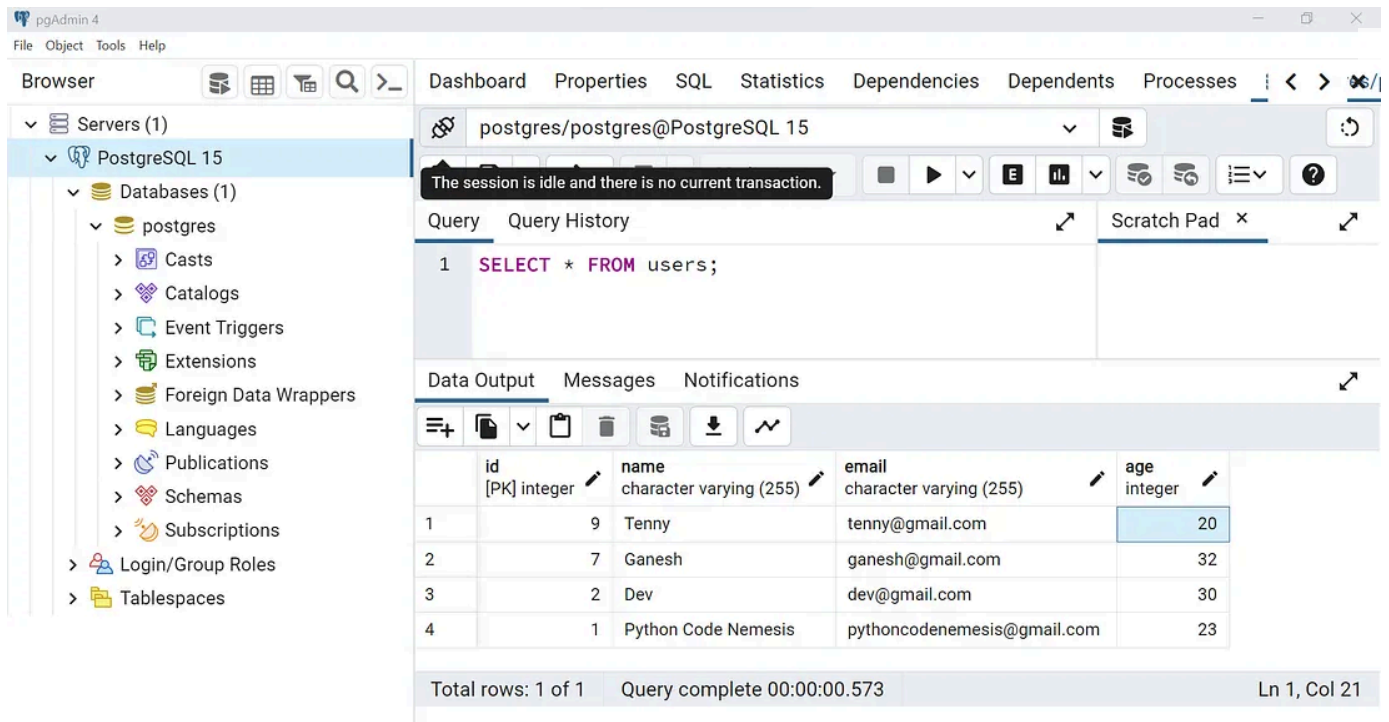
Execute the file with queries.py with:

```
python3 queries.py
```

The output will look like this:

```
PS C:\Users\soumi\Python_PostgreSQL_Database_Queries> python .\queries.py
1 Python Code Nemesis pythoncodenemesis@gmail.com 23
1 Python Code Nemesis
1 Python Code Nemesis pythoncodenemesis@gmail.com 23
23
PS C:\Users\soumi\Python_PostgreSQL_Database_Queries> python .\queries.py
SELECT QUERY 1
1 Python Code Nemesis pythoncodenemesis@gmail.com 23
SELECT QUERY 2
1 Python Code Nemesis
ORDER BY
1 Python Code Nemesis pythoncodenemesis@gmail.com 23
DISTINCT
23
WHERE
PS C:\Users\soumi\Python_PostgreSQL_Database_Queries>
```

I added some more entries to the users table. Now the users table looks like this:



Let's run the same query again:

```
PS C:\Users\...\Python_PostgreSQL_Database_Queries> python .\queries.py
1 Python Code Nemesis pythoncodenemesis@gmail.com 23
1 Python Code Nemesis
1 Python Code Nemesis pythoncodenemesis@gmail.com 23
23
9 Tenny tenny@gmail.com 20
SELECT QUERY 2
1 Python Code Nemesis
2 Dev
7 Ganesh
9 Tenny
ORDER BY
9 Tenny tenny@gmail.com 20
1 Python Code Nemesis pythoncodenemesis@gmail.com 23
2 Dev dev@gmail.com 30
7 Ganesh ganesh@gmail.com 32
DISTINCT
32
30
20
23
WHERE
7 Ganesh ganesh@gmail.com 32
```

JOINS

To join two tables, we'll need to define them as separate classes in our code. For example, let's say we have another table called `orders`:

```
CREATE TABLE orders (
    id INTEGER PRIMARY KEY,
    user_id INTEGER,
    product TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Then next:

The screenshot shows the pgAdmin 4 interface. In the left sidebar, the 'Servers' tree is expanded to 'PostgreSQL 15', and the 'Databases' tree is expanded to 'postgres'. The 'Query' tab is active, displaying the following SQL code:

```
1 CREATE TABLE orders (  
2     id INTEGER PRIMARY KEY,  
3     user_id INTEGER,  
4     product TEXT,  
5     FOREIGN KEY (user_id) REFERENCES  
6 );  
7
```

The 'Messages' tab is also active, showing the output: 'CREATE TABLE' and 'Query returned successfully in 112 msec.' The status bar at the bottom indicates 'Total rows: 0 of 0', 'Query complete 00:00:00.112', and 'Ln 7, Col 1'.

Next, add some data

The screenshot shows the pgAdmin 4 interface. In the left sidebar, the 'Servers' tree is expanded to 'PostgreSQL 15', and the 'Databases' tree is expanded to 'postgres'. The 'Query' tab is active, displaying the following SQL code:

```
1 SELECT * FROM orders
```

The 'Messages' tab is also active, showing the output: 'SELECT * FROM orders' and 'Query returned successfully in 302 msec.' The status bar at the bottom indicates 'Total rows: 0 of 0', 'Query complete 00:00:00.302', and 'Ln 1, Col 21'.

After adding some data will look like this

The screenshot shows the pgAdmin 4 interface. On the left, the 'Servers' tree shows 'PostgreSQL 15' with a database 'postgres'. The 'queries' tab is active, showing a query: `SELECT * FROM orders`. The 'Data Output' tab shows the results of the query as a table with 5 rows and 3 columns: `id` (integer, PK), `user_id` (integer), and `product` (text). The status bar at the bottom indicates 'Total rows: 0 of 0', 'Query complete 00:00:00.302', 'Changes staged: Added: 5', and 'Ln 1, Col 21'.

	id [PK] integer	user_id integer	product text
1+	10	678	Coach
2+	5	5	Charles a...
3+	47	8	Marks an...
4+	2	7	Chanel bag
5+	7	2	Gucci bag

Update the `queries.py` code:

```
class Order(Base):
    __tablename__ = 'orders'
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey('users.id'))
    product = Column(String)
```

This table has three columns: `id`, `user_id`, and `product`. `user_id` is a foreign key that references the `id` column in the `users` table

Creating tables with SQLAlchemy in Python

To map the foreign key column “column” in “table1” to the primary key column “column2” in “table2” using Python, you can modify your code as follows:

```
from sqlalchemy import Column, Integer, String, ForeignKey
from sqlalchemy.orm import relationship
class Table1(Base):
    __tablename__ = "table1"
    id = Column(Integer, primary_key=True)
    column = Column(String, ForeignKey("table2.column2"))
    table2 = relationship("Table2")
class Table2(Base):
    __tablename__ = "table2"
    id = Column(Integer, primary_key=True)
    column2 = Column(String)
```

Here, we define two SQLAlchemy model classes, “Table1” and “Table2”, with a one-to-many relationship between them. The foreign key column “column” in “Table1” is defined using the `ForeignKey` function, which specifies the name of the target column (“table2.column2”) that it references. The `relationship` function is used to define the corresponding relationship attribute in “Table1” that allows access to the related rows in “Table2”.

All the code for this tutorial is available here :

[GitHub - PythonCodeNemesis/Python_PostgreSQL_Database_Queries](https://github.com/PythonCodeNemesis/Python_PostgreSQL_Database_Queries)

github.com

That's it for this article! Feel free to leave feedback or questions in the comments. If you found this an exciting read, leave some claps and follow! Cheers!

[Python](#)[Python Programming](#)[Data](#)[Database](#)[Postgresql](#)[Follow](#)

Published in Technology Hits

4.4K followers · Last published 5 hours ago

We cover important, high-impact, informative, engaging stories on all aspects of technology. Subscribe to our content marketing strategy newsletter: <https://drmehmetiyildiz.substack.com/> Apply via: <https://digitalmehmet.com/contact> External: <https://illumination-curated.com>

[Follow](#)

Written by Python Code Nemesis

761 followers · 73 following

Everything python, DSA, open source libraries and more!

